

Coding Standards

Your projects and labs will be graded partially on conformance to these standards.

One of the most important aspects of modern programming is **readability**. To be able to read code written by other people and understand it has become much more important than having condensed, optimal code. Modern programming languages can optimize your code better than you can. So, the number one criteria for the way we write is for other human beings to read it. These standards are about **readability**, thus enhancing the maintainability your code when making future changes to it.

Indentation

The unit of indentation of JavaScript source code will be **four (4) spaces**. All text editors can do what is called *Soft Tabs*. When the tab key is pressed the editor enters spaces instead of the tab character. Your text editor should be set up to tab 4 spaces.

If a line of code is too long to fit on one line of the text editor (see Line Length standard below), the continuation of the line must be indented in by **eight (8) spaces** to visually emphasize that this line is a continuation.

Exception 1 When coding functions in a JavaScript file, the reserved word `function` must be written flush (no indentation) with the left side of the editing window.

Exception 2 When defining global variables/constants in the JavaScript file, the word `let` or `const` must be written flush (no indentation) with the left side of the editing window.

Basic Indentation

```
function exampleOne() {  
    var counter;  
    counter = 5;  
    document.write("It was 5!");  
}
```

Nested Indentation

```
function exampleOne(firstNumber, secondNumber) {  
    ....var counter;  
    ....var code;  
    ....  
    ....document.write(firstNumber, secondNumber);  
    ....var counter;  
    ....  
    ....counter = 5;  
    ....if (counter === 5) {  
        ....document.write("It was 5!");  
    ....}  
}
```

Inside of every set of

Code inside an if statement is indented four more spaces

Basic Continuation

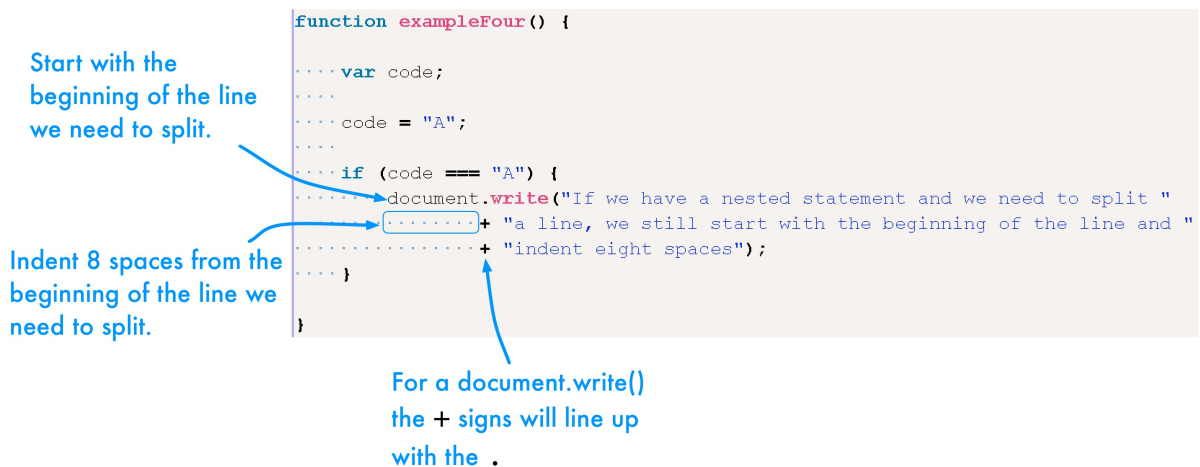
```
function exampleThree() {  
    ....document.write("Here's an example of text that is so long we need to split "  
    ....+ "it into more lines. We call that a line continuation. We use "  
    ....+ "eight spaces from the beginning of the line of code. If we need "  
    ....+ "to have more split lines they will all be indented "  
    ....+ "eight spaces.");  
}
```

Start with the beginning of the line we need to split.

Indent 8 spaces and start the continued line with the operator.

All continued lines will be indented 8 spaces.

Nested Continuation



Line Length

Avoid making any line of JavaScript code more than 80 characters. This improves readability. Sometimes this can't be avoided but when possible it is required to have 80 or fewer characters in a line of code. The course editor has been configured to show a vertical line at column 80 to help you with this standard:

Comments

Good documentation will be required in your labs and projects files. Here is a minimal list of what the instructor will be looking for. Quality counts!

1. The student's name, section, and email at the top of lab and project files.
 2. All functions that you write must have a comment block just above them describing what the function does. More details later when we get to functions.
 3. All difficult or confusing sections of code should have a preceding comment which explains the code.
-

Variable Declarations

Good variable naming and usage is very important when learning to program computers.

- Variable names must follow the naming conventions outlined in the *Naming Conventions*

standard below.

- All variables must be declared before they are used. Many programming languages require this as part of their syntax. However, JavaScript does not require this so you need do this intentionally.
- All `let` statements must be at the top of your JavaScript code *before any other statements*.
- Every variable must be declared on its own line.
- Variables should be declared using `let` and constants with `const`, never `var`.
- `let` statements must be terminated with a semicolon.
- There should be no unused variables left in the code. That means if a variable is not used for processing or displaying information, it's probably not needed. Variables that are declared and only assigned a value, are not necessary.

Naming Conventions

How things are named in a computer program is very important. In the early days of programming, names were very short and cryptic to save on RAM usage. You may have seen variables with names like "i", "fnme", and "cal_prc". These days our computers all have lots and lots of RAM and we can stop this old confusing habit. Here are the course rules for naming things:

- Variable names must be descriptive.
- Names must be created with the 26 upper and lower case letters (A..Z, a..z) and the 10 digits (0..9).
- Do not use the underscore `_`, the dollar sign `$`, or slashes `\ /` in variable names.
- Variables must start with a lower case letter.
- Abbreviations are not allowed—use complete words instead. What does "cal_prc" mean? Calculate Price? Calculate Percentage? California Practice? Calendar Pricing? What does "temp" mean? Temporary? Temperature? Temporary Employee? Names need to be very clear, for others and for you. You won't remember your own cryptic names in the future when you return to make enhancements to your code.
- Names that have more than one word (and most should) must use the "camel case" convention. The name starts with a lower case letter and then each new word in the name starts with an upper case letter, with no spaces between the words.

Examples:

```
1
```

```
// Good Names
```

```
2   let firstName;
3   let totalPrice;
4   let totalPriceForRedWidgets;
5   let customerCodeForNewNonUsRetailCustomer; // do programmers really do
    this? yes, we do!
```

Constants

All labs and projects must use constants for any values that are fixed and don't change while the program runs.

- Constants must be declared before variables. If the name of the constant is not completely self-explanatory, use a comment on the right side of that line.
- Constant names must NOT begin with a verb.
- Other than the numbers 1 and 0, numeric literals may not be used in any processing or calculation.
- Constants must be setup using the `const` statement, and not a `let` statement.
- Constants names will consist of:
 - Uppercase letters (and digits 0-9) only
 - Words must be separated by an underscore `_`
 - Must begin with an uppercase letter
 - Must not begin with an underscore nor a digit
- Constants are always assigned their values when they declared.
- Each constant must be defined on a separate line.

Constant Examples

```
1 // Declare Constants
2 const LOOP_MAXIMUM = 75;
3 const SALES_TAX = 0.05;
4
5 //Declare Variables ...
```

```
1 // Constants
2 const INITIAL_VALUE = 0;
3 const DAYS_IN_YEAR = 365;
4 const UPPER_BOUND = 100;
```

Global Variables & Global Constants

- Global variables/constants must follow the naming conventions outlined above.
 - All global variables/constants must be at the **top** of the JavaScript file.
 - Global variables/constants must **not** be indented, they are flush with the left margin.
-

Semicolons

Semicolons are used in JavaScript to terminate a statement. Most of the time they are not strictly required by JavaScript. However, they are required in a few places in JavaScript, so as a course standard (and one that is also a convention in the industry), the use of semicolons at the end of simple statements is required.

```
1 // Use of semicolons
2 let firstName; // note that let statements must end with semicolon
3 firstName = "Fred";
4
5 document.write(firstName);
6 discountTotal += 100;
7 flowerOne = temporaryFlowerHolder;
```

Statements

The way we write statements is critical for code that is readable and easy to understand. Here is how we're going to write JavaScript statements. Simple statements need to follow the rules below for this course:

- As emphasized just above, every simple statement must end with a semicolon.
- Every simple statement must be on its own line. This code will work but it fails the course standards.

Incorrect version

- On **line 2** there are TWO statements. JavaScript doesn't care but your instructor and future professional teammates will.

- On **line 3** the statement is missing the semicolon at the end. JavaScript won't remind you to put them there. You will just have to get in the habit.

```
1 // WRONG WAY!  
2 let name = "Fred"; let city = "Madison";  
3 document.write(name + ", " + city)
```

Correct version

```
1 // RIGHT WAY!  
2 let name;  
3 let city;  
4  
5 name = "Fred";  
6 city = "Madison";  
7  
8 document.write(name + ", " + city);
```

Whitespace

Whitespace greatly improves the readability of code. Here are the course standards for whitespace.

- Use blank lines liberally—your code shouldn't be all bunched together.
- After each section of your program, leave a blank line.
- A space should follow every comma.
- Most operators must have a space on either side.

Operators and spacing

```
1 // Operator spacing  
2 let firstName;  
3 let errorCount;  
4 let enteredNumbersSum;  
5  
6 firstName = "Fred";  
7 errorCount = 0;  
8  
9 errorCount += 3;  
10 enteredNumbersSum = 10 + 20;
```

Exception 1: The dot operator.

```
1 // No spaces around the dot
2 document.write("Hello World!");
```

Exception 2: Unary operators.

```
1 // No spaces around unary operators.
2 count++;
3 count--;
```

if Spacing

```
1 // There's a space after the word "if"
2 // There is a space after the closing ")"
3 // Code within the if statement is indented 4 spaces
4 /// There is a space before and after "else"
5 if (condition) {
6     ...
7 } else {
8     ...
9 }
```

Loop spacing

```
1 // There is a space after the keyword "while" and "for"
2 // There is a space after the closing ")"
3 // Code within the loop is indented 4 spaces
4 while (true) {
5     ...
6 }
7
8 for (counter = 0; counter > LOOP_LIMIT; counter++) {
9     ...
10 }
```

Strings

- All string literals must use double quotes. Do not use single quotes

Incorrect version

```
1 // WRONG WAY!
2 let state;
3
4 state = 'Wisconsin';
```


Correct version

```
1 // RIGHT WAY!  
2 let state;  
3  
4 state = "Wisconsin";
```

Numbers

- All calculations must be done with Number data types. No strings are allowed when doing mathematical operations.
- Decimals must be represented by a leading zero if the value is less than 1.

Incorrect version

```
1 // WRONG WAY!  
2 let tax;  
3  
4 tax = .55;
```

Correct version

```
1 // RIGHT WAY!  
2 let tax;  
3  
4 tax = 0.55;
```

Calculations

Calculations (and Comparisons) Must Use Variables (and Constants)

The results of all calculations must be assigned to a variable. No calculations may be performed inside a `document.write()` function except concatenation and `typeof`.

All calculations and comparisons must be performed with variables only. Use the names of the variables to reveal the meaning of the numbers for this application. No hard-coded numbers are allowed in calculations.

Calculations example

```

1 // Calculate into variables
2 let numberOne;
3 let numberTwo;
4 let total;
5
6 numberOne = 20;
7 numberTwo = 30;
8
9 total = numberOne + numberTwo;
10
11 document.write(total);
12
13 // NOT THIS!
14 document.write(numberOne + numberTwo);

```

Wrong

Hard-coded numbers are not allowed in calculations.

```

1 // WRONG Example
2 let total;
3
4 total = 12 * 3; //<-- Not allowed!

```

Correct

Variables and their values are used in all calculations.

```

1 //CORRECT Example
2 const MONTHS_IN_YEAR = 12; // <-- The 12 is now in a well-named constant
3
4 let totalAge; // <-- Better name for the variable
5 let yearsOld; // <-- Better name for the variable
6
7 yearsOld = 3;
8
9 totalAge = MONTHS_IN_YEAR * yearsOld; // <-- Now it's clear!

```

document.write() Usage

- Use the `document.write()` method only to output the content of variables, with, if needed, explanatory text concatenated with the variables.
- No mathematical operations may be done within the parentheses of a `document.write()`. Instead, do the math in a section of the program above the `document.write()` methods, with

the results stored into variables. Those variables are then used below within the `document.write()` method parentheses.

Compound statements

Compound statements are different than simple statements. They are statements that contain lists of other statements enclosed in curly braces `{ }`. This also includes while and for loops.

- Enclosed statements should be indented
- The opening brace `{` should be at the end of the line that starts the compound statement.
- The closing brace `}` should begin a line and should match the beginning of the compound statement.
- Braces `{ }` should be used on all enclosed statements, even if there is only one enclosed statement.

if Statement

```
1  const MINIMUM_AGE = 12;
2  let userAge;
3
4  userAge = prompt("Please enter your age.");
5
6  if (userAge < MINIMUM_AGE) {
7      alert("Sorry, you are not old enough.");
8  }
```

if/else Statement

```
1  const MINIMUM_AGE = 12;
2  let userAge;
3
4  userAge = prompt("Please enter your age.");
5
6  if (userAge < MINIMUM_AGE) {
7      alert("Sorry, you are not old enough.");
8  } else {
9      alert("You are old enough to continue.");
10 }
```

if/else if Statement

```
1  const MINIMUM_AGE = 12;
```

```

2   let userAge;
3
4   userAge = prompt("Please enter your age.");
5
6   if (userAge < MINIMUM_AGE) {
7       alert("Sorry, you are not old enough.");
8   } else if (userAge > MINIMUM_AGE) {
9       alert("You are old enough to continue.");
10  } else {
11      alert("You did not enter a number.");
12  }

```

while Loop

```

1   const LOOP_LIMIT = 10;
2   let loopCounter;
3
4   loopCounter = 0;
5
6   // There is a space after the keyword "while"
7   // There is a space after the closing ")"
8   // Code within the loop is indented 4 spaces
9   while (loopCounter < LOOP_LIMIT) {
10      document.write("Hello ten times.");
11      loopCounter += 1;
12  }

```

for Loop

```

1   const LOOP_LIMIT = 10;
2   let loopCounter;
3
4   // There is a space after the keyword "for"
5   // There is a space after the closing ")"
6   // Code within the loop is indented 4 spaces
7   for (loopCounter = 0; loopCounter < LOOP_LIMIT; loopCounter++) {
8       document.write("Hello ten times.");
9   }

```

```

1   // If the for statement extends past the 80 characters, extend it
2   // to the next line and indent
3   for (loopCounter = 0; loopCounter < LOOP_LIMIT;
4       loopCounter++) {
5       document.write("Hello ten times.");
6   }

```

Function Declarations

All programming logic must be inside a function. The only exception is global variables or global constants.

- All functions must be declared in a JavaScript file (the file with an extension of `.js`). No functions are allowed in the HTML document.
- When coding functions, the reserved word `function` must be written flush (no indentation) with the left side of the editing window.
- All new functions created by students must have names that begin with a **verb**. These names must be in **camel case**, and contain no abbreviated words.
- All functions must start with the `use strict;` directive
- All functions must be preceded by a comment block. The comment itself must begin with the phrase: "The purpose of this function is to " and then followed with a one or two sentence statement of what the function accomplishes.
- Functions may not be set up within other functions.
- All functions must be **declared before they are used**, so when a function calls another function, the called function must be physically located *above* it in the `.js` library file.
- The "controller" function must be located at the very *bottom* of the `js` library file. Your code will run without conformance with this standard so care must be taken to follow it.
- There should be no space between the name of a function and the `(` (opening parenthesis). There should be one space between the `)` (closing parenthesis) and the `{` (opening curly brace) that begins the body of the function.
- The statements comprising the body of the function itself are indented four spaces. The `}` (closing curly brace) is vertically aligned with the line containing the beginning of the declaration of the function (the "f" in the word "function".)

Function Example

```
1  /*
2     The purpose of this function is to
3     multiply two numbers.
4  */
5  function multiplyTwoNumbers() {
6      "use strict";
7      ...
8  }
```

The `return` Statement

If used, a return statement with a value must not use `( )` (parentheses) around the value. The return value expression must start on the same line as the return keyword in order to avoid semicolon insertion.

```
1 // Correct
2 return resultSum;
```

```
1 // Incorrect!
2 return (resultSum);
```